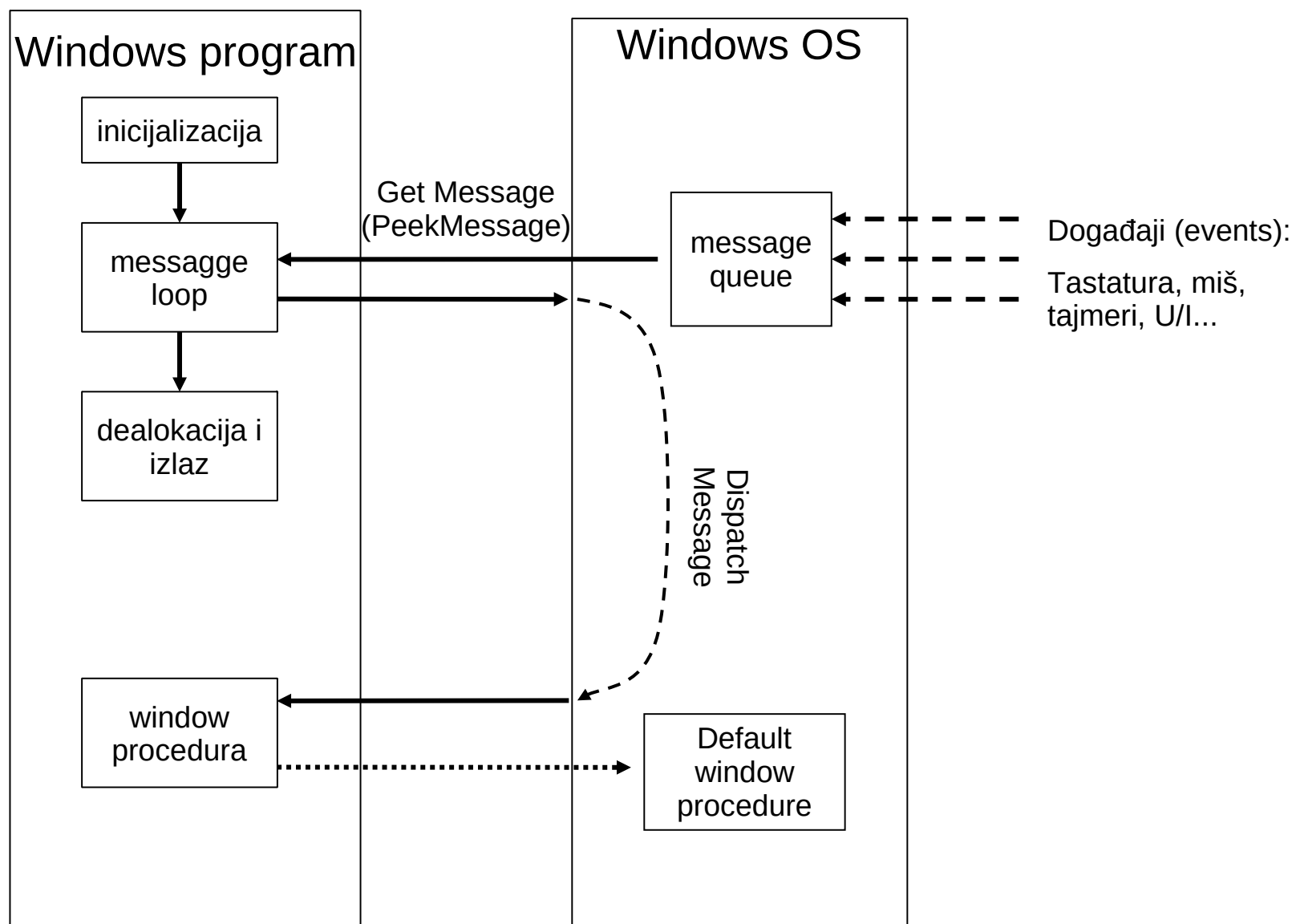


RI204

Windows programiranje

dr.sci. Edin Pjanić

- Korisnički ulazi
 - Tastatura
 - Miš
- Tajmer (engl. *timer*)



- Tastatura
 - ... je ulazni uređaj koji serijskom komunikacijom komunicira sa Windows OS šaljući paket bita koji opisuje taster koji je korisnik pritisnuo.
- Nakon što dekodira ovaj paket, Windows OS generiše odgovarajuće poruke za prozor (poziv window procedure).
- Poruke od tastature prima prozor koji je u fokusu. To znači da samo jedna window procedura, vezana za prozor koji trenutno ima fokus, prima poruke sa tastature.
- Poruke se ne šalju direktno u window proceduru već prvo idu u jedinstveni sistemski red poruka (system message queue), iz kojeg se prebacuju u window proceduru samo kada aplikacija završi sa procesiranjem prethodnog ulaza.
- Treba razlikovati pritisak tipke na tastaturi i karakter (znak koji se može ispisati), koji je rezultat tog pritiska.

- U vezi sa tastaturom, window procedura može primiti sljedeće poruke:
 - WM_KEYDOWN
 - Kada se pritisne bilo koji taster na tastaturi.
 - WM_KEYUP
 - Kada se otpusti bilo koji taster na tastaturi.
 - WM_CHAR
 - Kada je generisan znak (karakter) na tastaturi. `wparam` sadrži ASCII kod pritisnutog tastera. WM_CHAR je rezultat poziva funkcije `TranslateMessage`.
 - WM_SYSDEADCHAR, WM_SYSKEYUP i WM_SYSKEYDOWN
 - Isto kao u prethodim slučajevima samo u kombinaciji sa pritisnutim ALT tasterom
 - Procesiranje ovih poruka obično se prepušta default window proceduri (`DefWindowProc`)
 - Parametri:
 - `wparam`: kod (oznaka) tastera, značenje ovisi o poruci
 - `lparam`: dodatni podaci.

- Kodovi tastera (`wparam`)
 - Svim tasterima je asociran određen kod (scan code), ovisno od rasporeda tastera na tastaturi.
 - Windows OS konvertuje ovaj kod u tzv. virtualni kod tipke tastature (virtual key code) i isti prosljeđuje u window proceduru sa `WM_KEYDOWN` i `WM_KEYUP` porukom kao `wparam` parametar. Neke od vrijednosti su (simboličke konstante):
 - `VK_UP`, `VK_DOWN`, `VK_LEFT`, `VK_RIGHT`
 - Strelice na tastaturi za gore, dole, lijevo i desno
 - `VK_PRIOR`, `VK_NEXT`, `VK_END`, `VK_HOME`, `VK_ESCAPE`
 - Page up, page down, end, home i escape tasteri
 - `VK_CONTROL`, `VK_SHIFT`, ...

- lparam za WM_KEYDOWN, WM_KEYUP i WM_CHAR nosi 6 parametara upakovanih u 32 bita i to:

31	30	29	28	25	24	23	16	15	0
tran.	prev.	cntxt.	-	ext	.	scan code	rep. count.		

- 0 - 15 *repeat count*, broj auto-ponavljanja pritiska tipke u slučaju da korisnik drži pritisnutu tipku dok traje procesiranje
- 16 - 23 scan code, kod pritisnute tipke koji ovisi od proizvođača (OEM)
- 24 *extended flag*, TRUE ako je pritisnut taster sa enhanced tastature
- 29 *context code*, za WM_CHAR: 1 ako je pritisnut taster skupa sa ALT tasterom, inače: 0
- 30 prethodno stanje tipke: 0 - gore, 1 - dole. WM_KEYUP: 1
- 31 tranziciono stanje tipke: 1 - otpuštanje, 0 - pritiskanje,
 - za WM_KEYDOWN: 0 i za WM_KEYUP: 1
- Parametre je moguće provjeriti (otpakovati) pomoću bitwise operatora:

```
// Npr. pri primljenoj poruci WM_CHAR
if (lparam & 0x20000000)
{
    // Pritisnut je ALT skupa sa tipkom, uradi nesto
}
```

- Poruke se obično procesiraju unutar windows procedure u okviru dodatnog switch izraza:

```
switch (message)
case WM_KEYDOWN:
{
    switch (wparam)
    {
        case VK_HOME:
            // pritisnut HOME: idi na pocetak dokumenta;
            break ;

        case VK_END:
            // pritisnut END: idi na kraj dokumenta
            break ;

        case VK_PRIOR:
            // pritisnut PAGE UP: prijedji na prethodnu stranicu
            break ;
        // itd.
    }
    ...
}
```


- Poruka WM_CHAR služi za procesiranje karaktera
 - wparam parametar ima vrijednost ASCII koda pritisnutog tastera uvažavajući velika i mala slova (tj. ako korisnik drži pritisnut SHIFT taster ili ima aktiviran CAPS LOCK)
- Funkcija TranslateMessage vrši proces konverzije virtuelnog koda u validni karakter kod i prosljeđuje ga u red poruka kao dodatnu poruku sa kodom WM_CHAR pri pritisku neke od sljedećih tipki:
 - Bilo koji znak koji se može prikazati na ekranu (karakter)
 - BACKSPACE
 - ENTER (carriage return)
 - ESC
 - SHIFT+ENTER (linefeed)
 - TAB

Primjer

- Ukoliko je korisnik generisao veliko slovo A pomoću prethodno pritisnutog SHIFT tastera te zatim otpustio taster A pa taster SHIFT, windows procedura dobija sljedeće poruke:
 - WM_KEYDOWN sa wparam VK_SHIFT
 - WM_KEYDOWN sa wparam 0x41
 - WM_CHAR sa wparam 'A'
 - WM_KEYUP sa wparam 0x41
 - WM_KEYUP sa wparam VK_SHIFT

```
LRESULT CALLBACK WndProc (HWND hwnd, UINT message, WPARAM wParam, LPARAM lParam)
{
    static char tekst[100];
    PAINTSTRUCT ps;
    HDC hdc;
    switch (message)
    {
        case WM_CREATE:
            tekst[0] = '\\0';
            return 0 ;
        case WM_CHAR:
            {
                int pozicija = strlen(tekst);
                if ((wParam != '\\b') && (pozicija < 99))
                {
                    tekst[pozicija] = wParam;
                    tekst[pozicija + 1] = '\\0';
                }
                else
                {
                    if (pozicija > 0)
                        tekst[pozicija - 1]='\\0';
                }
                InvalidateRect (hwnd,NULL,TRUE);
                return 0;
            }
        case WM_PAINT:
            hdc = BeginPaint (hwnd, &ps) ;
            TextOut (hdc,0,0,tekst,strlen(tekst));
            EndPaint (hwnd,&ps);
            return 0;
        case WM_DESTROY:
            PostQuitMessage (0) ;
            return 0 ;
    }
    return DefWindowProc (hwnd, message, wParam, lParam) ;
}
```

- Standardni sistem prosljeđivanja poruka sa tastature prespor je za igre.
- Alternativa: umjesto da se čeka KEY poruka od windowsa unutar message loopa baziranog na PeekMessage funkciji, u svakom frame-u igre se može vršiti provjera stanja bitnih tastera pomoću funkcije `GetAsyncKeyState`:
 - `SHORT GetAsyncKeyState(int virtual_code);`
 - Funkcija provjerava stanje tastera `virtual_code` i da li je taster bio pritisnut od zadnjeg poziva funkcije. Povratna vrijednost ima značenje:
 - Ako je najviši bit postavljen na 1 tipka je trenutno pritisnuta.
 - Ako je najmanji bit postavljen na 1 tipka je bila pritisnuta od zadnjeg pozivanja ove funkcije, bez obzira da li je trenutno pritisnuta.
- Sljedeći makroi mogu poslužiti za test da li je tipka pritisnuta:

```
#define PRITISNUTO(vk_code) ((GetAsyncKeyState(vk_code) & 0x8000) ? 1 : 0)
#define BILO_PRITISNUTO(vk_code) ((GetAsyncKeyState(vk_code) & 0x0001) ? 1 : 0)
#define BILO_ILI_JE_PRITISNUTO(vk_code) ((GetAsyncKeyState(vk_code) & 0x8001) ? 1 : 0)
```

Primjer: kontrola kretanja lika tasterima

13

```
while(TRUE)
{
    DWORD vrijeme_pocetak;

    vrijeme_pocetak = GetTickCount();
    if (PeekMessage(&msg, NULL, 0, 0, PM_REMOVE))
    {
        if (msg.message == WM_QUIT) break;
        TranslateMessage(&msg);
        DispatchMessage(&msg);
    }
    anim_ulaz();
    anim_sracunaj();
    anim_crtaj();
    while((GetTickCount() - vrijeme_pocetak) < PAUZA);
}
```

```
void anim_ulaz()
{
    if (PRITISNUTO(VK_RIGHT))
        lik.smjer = 1;

    else if (PRITISNUTO(VK_LEFT))
        lik.smjer = -1;
}
```

Funkcija za tretman ulaza

GetTickCount() vraća broj milisekundi od pokretanja sistema.

- Window procedura dobija poruke vezane za aktivnost miša svaki put kada se miš pomjera iznad prozora ili kada korisnik izvrši klik unutar prozora
 - Kada se miš pomjera iznad prozora prosljeđuje se poruka WM_MOUSEMOVE a na klik miša prozor dobija tri tipa poruka:
 - Lijevo: WM_LBUTTONDOWN, WM_LBUTTONUP WM_LBUTTONDBLCLK
 - Srednje: WM_MBUTTONDOWN, WM_MBUTTONUP WM_MBUTTONDBLCLK
 - Desno: WM_RBUTTONDOWN, WM_RBUTTONUP WM_RBUTTONDBLCLK
- U svim porukama parametri su sljedeći:
 - `lparam` pakuje informaciju o **X** i **Y** koordinati pozicije miša i to:
 - `y = HIWORD(lparam); x = LOWORD(lparam);`
 - `wparam` indicira status tipki miša te SHIFT i CTRL tastera i to sa sljedećim bit maskama:
 - `MK_LBUTTON, MK_RBUTTON, MK_MBUTTON, MK_SHIFT, MK_CONTROL`
 - Za provjeru stanja pojedine tipke miša koriste se bitski operatori.
 - Npr. `wparam & MK_CONTROL` - testira da li je skupa sa mišem pritisnut CTRL taster

- WM_MOUSEMOVE ne garantuje da će poruke za svaku poziciju miša doći u window proceduru (ovisi od hardware-a i brzine procesiranja poruka u window proceduri)
- Za primanje duplih klikova potrebno je da prozor bude klase CS_DBLCLKS
- Moguća je situacija u kojoj prozor dobije WM_LBUTTONDOWN a da ne dobije WM_LBUTTONUP poruku i obrnuto.
- Moguće je podesiti sistem da prima poruke i kad je miš izvan klijentskog područja (izvan prozora).
 - Da bi prozor primao poruke od miša koji je van svog klijent područja potrebno je izvršiti poziv funkcije:
 - SetCapture(hwnd);
 - Nakon toga Windows OS preusmjerava sve poruke u vezi sa mišem u dotični prozor (hwnd). Primljene koordinate miša su još uvijek u odnosu na klijentski koordinatni početak. Za vraćanje normalne kontrole potrebno je pozvati funkciju:
 - ReleaseCapture();

- Za primanje periodičnih notifikacija o isteklom vremenu u Windows OSu moguće je koristiti timer.
 - Timer se može postaviti da šalje poruku prozoru u redovnim vremenskim intervalima
 - Poruke za timer smještaju se u standardni red poruka i niskog su prioriteta
 - Može biti samo jedna timer poruka u redu poruka prozora
 - Rezolucija timer-a je 10 msec
 - Ne postoji garancija da će poruke biti dostavljene tačno na vrijeme

- Za kreiranje tajmera može se koristiti funkcija:

```
UINT SetTimer( HWND hWnd, UINT nIDEvent, UINT uElapse, TIMERPROC lpTimerFun);
```

- Gdje je:
 - hWnd prozor koji će primati poruke od kreiranog tajmera
 - nIDEvent identifikator (ID, oznaka) tajmera
 - uElapse interval slanja poruka u msec
 - lpTimerFun opcionalna funkcija kojoj se prosljeđuje poziv u slučaju da se tajmer poruka tretira van windows procedure
- Prozor u redovnim intervalima zatim dobija poruku WM_TIMER sa wParam parametrom postavljenim na ID tajmera koji je generisao poruku
- Tajmer sa identifikatorom nIDEvent se zaustavlja pozivom funkcije:
 - BOOL KillTimer(HWND hWnd, UINT uIDEvent);

Primjer (iz knjige C. Petzold)

```
LRESULT CALLBACK WndProc (HWND hwnd, UINT message, WPARAM wParam, LPARAM lParam)
{
    static BOOL fFlipFlop = FALSE ;
    HBRUSH      hBrush ;
    HDC          hdc ;
    PAINTSTRUCT ps ;
    RECT         rc ;
    switch (message)
    {
        case WM_CREATE:
            SetTimer (hwnd, ID_TIMER, 1000, NULL) ;
            return 0 ;
        case WM_TIMER :
            MessageBeep (-1) ;
            fFlipFlop = !fFlipFlop ;
            InvalidateRect (hwnd, NULL, FALSE) ;
            return 0 ;
        case WM_PAINT :
            hdc = BeginPaint (hwnd, &ps) ;
            GetClientRect (hwnd, &rc) ;
            hBrush = CreateSolidBrush (fFlipFlop ? RGB(255,0,0) : RGB(0,0,255)) ;
            FillRect (hdc, &rc, hBrush) ;
            EndPaint (hwnd, &ps) ;
            DeleteObject (hBrush) ;
            return 0 ;
        case WM_DESTROY :
            KillTimer (hwnd, ID_TIMER) ;
            PostQuitMessage (0) ;
            return 0 ;
    }
    return DefWindowProc (hwnd, message, wParam, lParam) ;
}
```